TSYP14 Technical Challenge – City Brain Convergence

Interface Specification Book

What the City Brain layer expects from
the Twin and Guardian layers

Version 0.1 – draft for review by the triplet

15 October 2026

Prepared by	City Brain layer team
For	Twin layer team, Guardian layer team
Status	Proposal. Every item is open to discussion until the interface freeze.
Scope	Communication, security and message contents between the three layers

Contents

1 Introduction **3**
1.1 Purpose . . . 3
1.2 Decisions already taken . . . 3
1.3 How to read the requirements . . . 3
1.4 Changes to this document . . . 3

2 System overview **3**
2.1 Who sends what to whom . . . 4
2.2 What the City Brain does with your data . . . 4

3 Transport: MQTT **5**

4 Security **5**
4.1 Connection security (TLS, accounts, permissions) . . . 5
4.2 Message security (HMAC, timestamp, id) . . . 6

5 Message format **7**
5.1 Common body fields . . . 7
5.2 Naming and units . . . 7

6 Requirements on the Twin layer **7**
6.1 City description . . . 8
6.2 Sensor readings . . . 8
6.3 Emergency calls . . . 9
6.4 Commands and acknowledgements . . . 9
6.5 Scenarios and status . . . 10

7 Requirements on the Guardian layer **10**
7.1 Trust verdicts . . . 10
7.2 Quarantine and status . . . 11

8 What the City Brain layer provides **12**

9 Error handling **12**

10 Mocks and integration tests **13**
10.1 Shared examples and mocks . . . 13
10.2 Interface tests . . . 13

11 Open points to agree **14**

A Topic summary **14**

B HMAC reference implementation and test vector **15**

B.1 Python 15

B.2 Test vector 15

1 Introduction

1.1 Purpose

This book describes, as precisely as possible, **what the City Brain layer needs to receive** from the Twin and Guardian layers, and **what it sends back**. Its goal is that each team can build its layer, and its mocks of the other layers, independently, and that the three layers work together the first time they are connected.

It covers only what crosses the boundary between layers. How each team builds the inside of its layer is free.

1.2 Decisions already taken

- **MQTT** is the communication protocol between all layers.
- **TLS** protects every connection; every message carries an **HMAC**, a **timestamp** and a **unique id**.
- Each layer runs on its own with **mocks** of the other two (Phase 1 requirement R1).

1.3 How to read the requirements

Each requirement has an identifier and a level:

MUST	Required for the layers to work together. Without it, integration fails.
SHOULD	Strongly recommended. It can be skipped only with a good reason, agreed by the triplet.
MAY	Optional. Useful if time allows.

Identifiers: `TR` transport, `SEC` security, `MSG` message format, `TW` Twin layer, `GD` Guardian layer, `BR` City Brain layer, `IT` integration tests.

1.4 Changes to this document

- This version is a **draft**. Comments are welcome until the interface freeze date agreed by the triplet.
- After the freeze, any change is announced in the triplet channel, written in this document with a new version number, and reflected in the shared `INTEGRATION.md`.
- No layer changes a topic name or a message field silently.

2 System overview

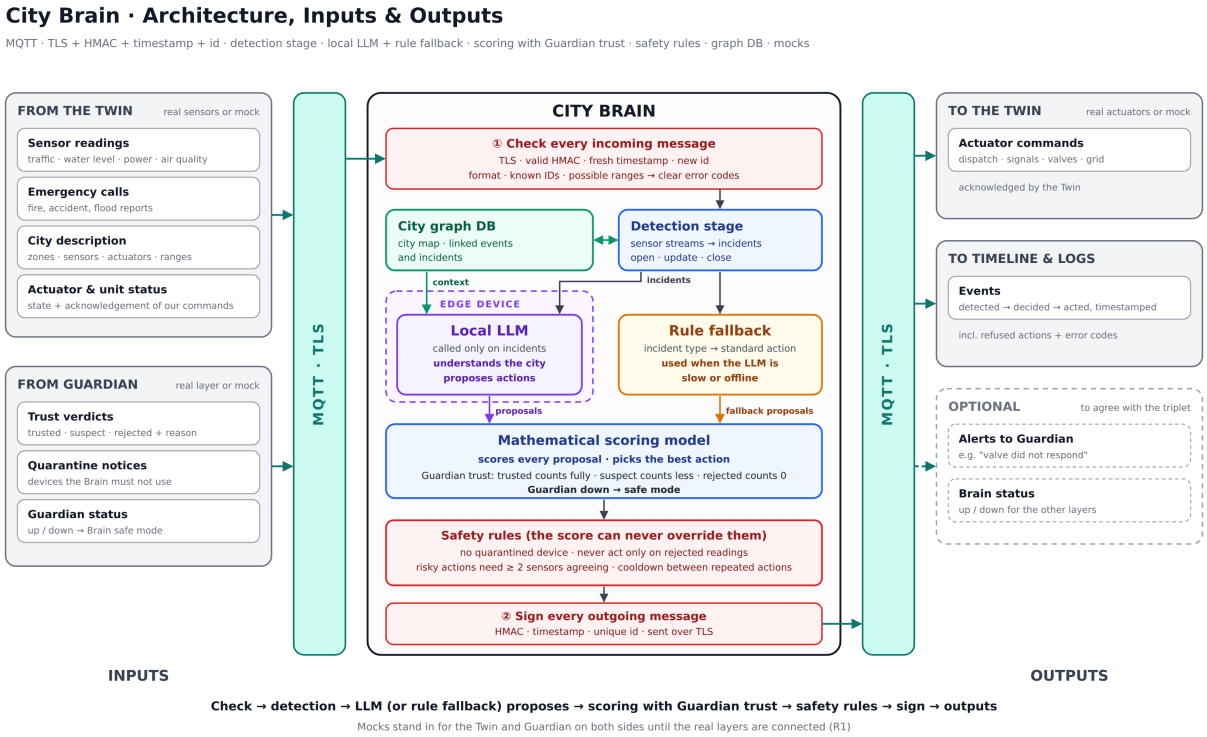


Figure 1: City Brain architecture: inputs from the Twin and Guardian, outputs to the Twin and the timeline. All exchanges go through the MQTT broker over TLS.

2.1 Who sends what to whom

From	To	Content
Twin	Brain, Guardian	City description (zones, sensors, actuators, units)
Twin	Brain, Guardian	Sensor readings (traffic, water level, power load, air quality)
Twin	Brain, Guardian	Emergency calls
Twin	Brain	Actuator and unit state, acknowledgement of each command
Guardian	Brain	Trust verdicts on sensors / readings
Guardian	Brain, Twin	Quarantine notices for devices
Brain	Twin	Actuator commands (dispatch, traffic signals, valves, grid)
Brain	Timeline	Events (incident detected, decision, action, refusal)
All layers	All layers	Status: online / offline

2.2 What the City Brain does with your data

So that each team understands why each requirement exists:

1. Every incoming message is **checked**: TLS, HMAC, timestamp, unique id, format, known IDs and possible range. Rejected messages are logged with an error code (Section 9).
2. A **detection stage** turns the sensor streams into **incidents** (flood, fire, accident, traffic jam, power problem).
3. A local LLM and a rule-based fallback **propose actions**; a scoring model picks the best one, giving each reading a weight based on **Guardian's verdict**.

4. **Safety rules** block any action on a quarantined device, any action based only on rejected readings, and risky actions confirmed by a single sensor.
5. The chosen action is sent to the Twin as a **signed command**, and the Brain checks the acknowledgement.

3 Transport: MQTT

ID	Requirement	Level
TR-1	All layers exchange data only through **one shared MQTT broker** (Mosquitto proposed).	**MUST**
TR-2	Every client supports **MQTT 3.1.1**. MQTT 5 features are not required, so that embedded clients (e.g. ESP32 with real sensors) can connect.	**MUST**
TR-3	The broker address, port, credentials and topic prefix are read from a **configuration file**, never hard-coded.	**MUST**
TR-4	Each layer connects with a **fixed client id**: `twin`, `guardian`, `brain` (plus a suffix for extra clients, e.g. `twin-gw1`).	**SHOULD**
TR-5	QoS and retained flags follow the topic table in Annex A.	**MUST**
TR-6	Each layer publishes its status on `{layer}/status`: `online` (retained) when connected, and sets a **Last Will** `offline` (retained) so others are told when it disconnects.	**MUST**
TR-7	The broker runs on a machine connected to the **triplet's own router or hotspot**, never on the venue Wi-Fi.	**MUST**
TR-8	A client that loses the connection reconnects automatically and re-subscribes.	**MUST**

4 Security

4.1 Connection security (TLS, accounts, permissions)

ID	Requirement	Level
SEC-1	The broker accepts connections only over **TLS** (port 8883). Plain port 1883 is disabled for the demo.	**MUST**
SEC-2	The triplet generates **one certificate authority (CA)**; every client verifies the broker certificate against it.	**MUST**
SEC-3	Anonymous access is disabled. Each layer has **its own username and password** (or client certificate).	**MUST**
SEC-4	The broker enforces the **topic permissions (ACL)** of Table 3: each layer can publish only on its own topics.	**MUST**
SEC-5	Passwords, keys and certificates are kept in files excluded from Git (`.env`, `secrets/`), never in the repository.	**MUST**
SEC-6	Client certificates (mutual TLS) instead of passwords.	**MAY**

Account	May publish on	May subscribe to
`twin`	`city/#`, `twin/#`	`brain/cmd/#`, `guardian/quarantine/#`, `+/status`
`guardian`	`guardian/#`	`city/#`, `twin/#`, `brain/#`, `+/status`
`brain`	`brain/#`	`city/#`, `twin/#`, `guardian/#`, `+/status`
`timeline`	– (read only)	`#`

Table 3: Topic permissions per account. A compromised Twin client cannot publish fake Guardian verdicts, and only the Brain can send commands.

4.2 Message security (HMAC, timestamp, id)

TLS protects the connection, but inside the broker a message is readable again. The HMAC proves that each message was written by a holder of the shared secret and was not changed on the way, even through the broker.

Envelope. To avoid any difference in how languages format JSON (spaces, key order, numbers), the HMAC is computed over the **exact text** of the message body. Every MQTT payload is therefore an **envelope**:

```
{
  "v": 1,
  "body": "<the message, as a JSON text string>",
  "hmac": "<HMAC-SHA256 of the body text, lowercase hex>"
}
```

The receiver verifies the HMAC on the `body` text first, and only then parses the body.

ID	Requirement	Level
SEC-7	Every payload uses the envelope above, with `"v": 1`.	**MUST**
SEC-8	`hmac` = HMAC-SHA256(key, UTF-8 bytes of `body`), written as 64 lowercase hexadecimal characters.	**MUST**
SEC-9	**One secret key per pair of layers**: Twin–Brain, Twin–Guardian, Guardian–Brain. Keys are at least 32 random bytes.	**MUST**
SEC-10	Messages published to several layers (readings, calls, city description, status) are signed with the key of each receiving pair, in the field `hmacs`: `{"brain": "...", "guardian": "..."}` instead of `hmac`.	**MUST**
SEC-11	The receiver rejects a message whose HMAC is missing or wrong (error `E_AUTH`).	**MUST**
SEC-12	Every body has a `timestamp` (Section 5). The receiver rejects a message more than **10 seconds** older or newer than its own clock (error `E_STALE`).	**MUST**
SEC-13	Every body has a `id` unique for its sender. The receiver remembers the ids seen in the last 60 seconds and rejects repeats (error `E_REPLAY`).	**MUST**

ID	Requirement	Level
SEC-14	All machines, including sensor boards, synchronise their clocks with **NTP** from the triplet router or the broker machine, so that the 10-second window works.	**MUST**
SEC-15	New keys are generated before the final event if any key was shared in a chat or committed by mistake.	**SHOULD**

What this does not cover. These measures stop outsiders and tampering between layers. They do not detect a sensor that sends false values with a valid HMAC, because the Twin legitimately signs its own readings. Detecting that is the Guardian layer's job (trust verdicts), and the City Brain adds its own checks (no action on rejected readings, confirmation from more than one sensor).

5 Message format

5.1 Common body fields

Every message body, in every direction, starts with these fields:

Field	Type	Meaning	Level
`id`	string	Unique message id for the sender, e.g. `twin-000182`	**MUST**
`timestamp`	string	Time of sending, ISO 8601 in UTC with milliseconds, e.g. `2026-11-14T10:22:03.120Z`	**MUST**
`source`	string	Sending layer: `twin`, `guardian` or `brain`	**MUST**
`correlation_id`	string	Id of the original message this one results from (a verdict on a reading, an acknowledgement of a command). Lets every layer and the timeline link cause and effect.	**SHOULD**

5.2 Naming and units

ID	Requirement	Level
MSG-1	All messages are JSON, UTF-8. Field names are `snake_case`.	**MUST**
MSG-2	IDs use lowercase letters, digits and dashes, and are stable during a run: zones `port`, `z3`; sensors `water-level-04`; actuators `valve-port-up`; units `ambulance-2`.	**MUST**
MSG-3	Every sensor, actuator and unit id used in any message is declared in the city description (TW-1).	**MUST**
MSG-4	Values use the units of Table 6.	**MUST**
MSG-5	Unknown extra fields are ignored by the receiver, so that a layer can add information without breaking the others.	**MUST**
MSG-6	Message size stays under 16 KB.	**SHOULD**

6 Requirements on the Twin layer

Sensor type (`type`)	**Unit** (`unit`)	**Meaning**
`water_level`	`m`	Water height above the reference level
`traffic_speed`	`km/h`	Average vehicle speed
`traffic_density`	`veh/km`	Vehicles per km of road
`power_load`	`kW`	Electrical load
`air_quality`	`ug/m3`	PM2.5 concentration
`smoke`	`0-1`	Smoke detector level
`temperature`	`degC`	Air temperature

Table 6: Sensor types and units (the Twin team may add types; they must appear in the city description).

6.1 City description

ID	**Requirement**	**Level**
TW-1	The Twin publishes the **city description** on `twin/meta`, **retained**, QoS 1, at start-up and whenever it changes.	**MUST**
TW-2	It lists every **zone** with its neighbours (`adjacent`), every **sensor** with type, zone, unit, possible range (`min`, `max`) and alert level, every **actuator** with the zones it controls and the actions it accepts, and every **unit** (ambulance, fire, police, rescue).	**MUST**
TW-3	Zone `type` and `exposure` (e.g. hospital, school) are included.	**SHOULD**

Listing 1: Body of the city description (shortened)

```
{
  "id": "twin-meta-001", "timestamp": "2026-11-14T09:00:00.000Z", "source": "twin",
  "zones": [
    {"zone_id": "port", "name": "Port", "type": "port", "exposure": "high",
     "adjacent": ["docks", "z4"]}
  ],
  "sensors": [
    {"sensor_id": "water-level-04", "zone_id": "port", "type": "water_level",
     "unit": "m", "min": 0.0, "max": 4.0, "alert": 2.0}
  ],
  "actuators": [
    {"actuator_id": "valve-port-up", "type": "valve", "zone_id": "port",
     "controls": ["port", "docks"], "actions": ["open", "close"]}
  ],
  "units": [
    {"unit_id": "ambulance-2", "type": "ambulance", "home_zone": "z3"}
  ]
}
```

6.2 Sensor readings

ID	Requirement	Level
TW-4	Readings are published on `city/{zone_id}/sensor/{type}/{sensor_id}`, QoS 0.	**MUST**
TW-5	The body contains `sensor_id`, `zone_id`, `type`, `value` (number) and `unit`.	**MUST**
TW-6	Each sensor publishes at least **every 5 seconds**.	**MUST**
TW-7	Each sensor publishes every second.	**SHOULD**
TW-8	Real sensors are published through a gateway or board that applies the same envelope, HMAC, timestamp and id rules.	**MUST**

Listing 2: Body of a sensor reading

```
{"id": "twin-000182", "timestamp": "2026-11-14T10:22:03.120Z", "source": "twin",
 "sensor_id": "water-level-04", "zone_id": "port",
 "type": "water_level", "value": 2.4, "unit": "m"}
```

6.3 Emergency calls

ID	Requirement	Level
TW-9	Calls are published on `city/{zone_id}/call`, QoS 1, with `call_id`, `zone_id` and `category` (`fire`, `accident`, `flood`, `medical`, `other`).	**MUST**
TW-10	Calls include a short free-text description in `text` (max. 280 characters).	**SHOULD**

Listing 3: Body of an emergency call

```
{"id": "twin-000190", "timestamp": "2026-11-14T10:22:05.004Z", "source": "twin",
 "call_id": "call-0012", "zone_id": "z3", "category": "fire",
 "text": "Smoke coming from the substation"}
```

6.4 Commands and acknowledgements

ID	Requirement	Level
TW-11	The Twin subscribes to `brain/cmd/#` and executes valid commands (format in BR-1).	**MUST**
TW-12	A command with an `id` already executed is **not executed again** (MQTT QoS 1 can deliver a message twice).	**MUST**
TW-13	For every command, the Twin publishes an **acknowledgement** on `twin/actuator/{actuator_id}` (or `twin/unit/{unit_id}`) within **2 seconds**, QoS 1, with `correlation_id` = the command `id`, `result` (`done`, `rejected`, `failed`), the new `state` and, if not done, a `reason`.	**MUST**

ID	Requirement	Level
TW-14	The Twin also publishes the actuator state on the same topic whenever it changes, retained.	**SHOULD**
TW-15	Unit status (`free`, `en_route`, `on_scene`, `returning`) and current zone are published on `twin/unit/{unit_id}`, retained, whenever they change.	**MUST**
TW-16	Actions have a **visible effect on later readings** (valve opened → water level stops rising; signals retimed → traffic speed rises), so that the Brain can check its action worked.	**SHOULD**
TW-17	The Twin refuses commands to a device quarantined by Guardian, with `result: rejected`, `reason: quarantined`.	**SHOULD**

Listing 4: Body of an acknowledgement

```
{"id": "twin-000205", "timestamp": "2026-11-14T10:22:04.410Z", "source": "twin",
 "correlation_id": "brain-000042",
 "actuator_id": "valve-port-up", "result": "done", "state": "open_80"}
```

6.5 Scenarios and status

ID	Requirement	Level
TW-18	The Twin publishes `twin/status` with Last Will (TR-6).	**MUST**
TW-19	The Twin can play each Test Book scenario from one command, including **faulty or fake readings** (stuck value, spike, false flood), which are still correctly signed.	**SHOULD**

7 Requirements on the Guardian layer

7.1 Trust verdicts

ID	Requirement	Level
GD-1	Guardian publishes a verdict per sensor on `guardian/verdict/{sensor_id}`, **retained**, QoS 1.	**MUST**
GD-2	The body contains `sensor_id`, `verdict` (`trusted`, `suspect`, `rejected`) and `reason` (code from Table 13).	**MUST**
GD-3	When the verdict is about a specific reading, `correlation_id` is that reading's `id`.	**SHOULD**
GD-4	A new verdict is published **as soon as it changes**, and at most **10 seconds** after a faulty or fake reading appears.	**MUST**
GD-5	The current verdict of every sensor is re-published at least every 10 seconds, so the Brain knows it is still valid.	**SHOULD**

ID	Requirement	Level
GD-6	A sensor that Guardian isolates is published with `verdict: rejected`, `reason: isolated`.	**MUST**

Reason code	Meaning
`ok`	Nothing wrong found
`out_of_range`	Value outside the possible range
`stuck_value`	Value not changing when it should
`spike`	Sudden jump not explained by neighbours
`drift`	Slow deviation from the sensor's normal behaviour
`inconsistent`	Contradicts neighbouring sensors
`replay`	Old message sent again
`bad_signature`	HMAC missing or wrong
`flooding`	Device sends far too many messages
`isolated`	Sensor isolated by Guardian
`other`	Any other reason (details in `details`)

Table 13: Proposed verdict reason codes.

Listing 5: Body of a verdict

```
{"id": "guardian-004521", "timestamp": "2026-11-14T10:22:03.900Z", "source": "guardian",
 "correlation_id": "twin-000182",
 "sensor_id": "water-level-04", "verdict": "rejected", "reason": "out_of_range"}
```

7.2 Quarantine and status

ID	Requirement	Level
GD-7	Quarantine notices are published on `guardian/quarantine/{device_id}`, **retained**, QoS 1, with `device_id`, `state` (`quarantined` or `released`) and `reason`.	**MUST**
GD-8	Guardian publishes `guardian/status` with Last Will (TR-6). When Guardian is offline, the Brain switches to safe mode.	**MUST**
GD-9	Guardian subscribes to `brain/alert` to receive the Brain's alerts (e.g. an actuator that does not respond).	**MAY**

Listing 6: Body of a quarantine notice

```
{"id": "guardian-004530", "timestamp": "2026-11-14T10:23:10.000Z", "source": "guardian",
 "device_id": "valve-port-main", "state": "quarantined", "reason": "inconsistent"}
```

8 What the City Brain layer provides

These are our commitments, so that the other teams can build their mocks of the Brain.

ID	Commitment	Level
BR-1	Commands are published on `brain/cmd/{actuator_id}` or `brain/cmd/{unit_id}`, QoS 1, with `actuator_id` or `unit_id`, `action` (one of the actions declared in the city description, or `dispatch` for units), `params`, `incident_id` and a short `reason`.	**MUST**
BR-2	The Brain never sends a command to a quarantined device and never acts only on rejected readings.	**MUST**
BR-3	The Brain publishes its events for the timeline on `brain/event`: incident opened / updated / closed, decision, command sent, acknowledgement received, action refused.	**MUST**
BR-4	The Brain publishes `brain/status` with Last Will (TR-6).	**MUST**
BR-5	Every message the Brain rejects is reported on `brain/error` with the error code, the topic and the message `id`, so partners can debug quickly.	**SHOULD**
BR-6	The Brain publishes alerts on `brain/alert` (e.g. `actuator_not_responding`).	**MAY**

Listing 7: Body of a command

```
{"id": "brain-000042", "timestamp": "2026-11-14T10:22:04.020Z", "source": "brain",
 "correlation_id": "twin-000182",
 "actuator_id": "valve-port-up", "action": "open", "params": {"percent": 80},
 "incident_id": "inc-0031", "reason": "flood at port"}
```

Listing 8: Body of a unit dispatch

```
{"id": "brain-000043", "timestamp": "2026-11-14T10:22:06.500Z", "source": "brain",
 "correlation_id": "twin-000190",
 "unit_id": "fire-1", "action": "dispatch", "params": {"zone_id": "z3"},
 "incident_id": "inc-0032", "reason": "fire in zone z3"}
```

Listing 9: Body of a timeline event

```
{"id": "brain-000044", "timestamp": "2026-11-14T10:22:04.025Z", "source": "brain",
 "correlation_id": "twin-000182",
 "event": "command_sent", "incident_id": "inc-0031",
 "summary": "flood at port -> open valve-port-up 80%"}
```

9 Error handling

Every layer applies the same checks to what it receives, and uses the same error codes in its logs:

Code	Rejected when	Example
`E_FORMAT`	Envelope or body is not valid JSON, or a required field is missing or has the wrong type	Reading without `value`
`E_AUTH`	HMAC missing or wrong	Message changed in transit
`E_STALE`	Timestamp more than 10 s away from the receiver's clock	Old message, or clock not synchronised
`E_REPLAY`	`id` already received in the last 60 s	Captured message sent again
`E_UNKNOWN_ID`	Sensor, zone, actuator or unit not in the city description	`valve-x9` does not exist
`E_RANGE`	Value outside the declared `min`/`max`	Water level −5 m

A rejected message is **never processed further**. It is logged with its code, and shown on the timeline.

10 Mocks and integration tests

10.1 Shared examples and mocks

- The example messages of this book are stored in the triplet repository as JSON files (`docs/interface/examples/`). Every team builds its mocks from these files, so that mocks and real layers match.
- Each team runs its layer alone with mocks of the other two (R1), using the same broker configuration, topics, envelope and HMAC.
- The HMAC test vector of Annex B is checked by every team's code before the first joint run.

10.2 Interface tests

These tests check the interfaces only. They come before the functional Test Book scenarios.

ID	Test	Expected result
IT-1	Brain starts after the Twin	Brain receives the retained city description
IT-2	Twin publishes a valid reading	Accepted by Brain and Guardian
IT-3	A reading with a wrong HMAC	Rejected, `E_AUTH`
IT-4	A reading 30 s old	Rejected, `E_STALE`
IT-5	The same reading sent twice	Second one rejected, `E_REPLAY`
IT-6	A reading from an undeclared sensor	Rejected, `E_UNKNOWN_ID`
IT-7	Guardian rejects a fake flood reading	Brain sends no command, logs the reason
IT-8	Guardian quarantines a valve	Brain uses another valve or none

ID	Test	Expected result
IT-9	Brain sends a command	Twin acknowledges within 2 s; a duplicate is not executed
IT-10	Guardian is stopped	`guardian/status = offline`; Brain switches to safe mode
IT-11	Twin is stopped	`twin/status = offline`; Brain and Guardian log it
IT-12	A client without credentials, or publishing on another layer's topic	Refused by the broker

11 Open points to agree

#	Point	With
1	Final topic names and the envelope format (Section 4.2)	All
2	Which machine hosts the broker; who creates the CA, certificates and accounts	All
3	How the three HMAC keys are generated and shared	All
4	Timestamp window (10 s) and id memory (60 s)	All
5	Sensor types, units and reading frequency of the real sensors	Twin
6	Actions accepted by each actuator type; unit types	Twin
7	Whether emergency calls include free text	Twin
8	Verdict per sensor or per reading; reason codes	Guardian
9	Whether `brain/alert` is used	Guardian
10	Who builds the shared timeline screen (it subscribes to all topics)	All
11	Interface freeze date and first joint run date	All

A Topic summary

Topic	Publisher	Subscribers	QoS	Retained
`twin/meta`	Twin	Brain, Guardian	1	yes
`city/{zone}/sensor/{type}/{sensor}`	Twin	Brain, Guardian	0	no
`city/{zone}/call`	Twin	Brain, Guardian	1	no
`twin/actuator/{actuator}`	Twin	Brain, Guardian	1	state: yes
`twin/unit/{unit}`	Twin	Brain	1	yes
`guardian/verdict/{sensor}`	Guardian	Brain	1	yes
`guardian/quarantine/{device}`	Guardian	Brain, Twin	1	yes
`brain/cmd/{actuator or unit}`	Brain	Twin	1	no
`brain/event`	Brain	Timeline	0	no
`brain/error`	Brain	All	0	no
`brain/alert`	Brain	Guardian	1	no

Topic	Publisher	Subscribers	QoS	Retained
{twin,guardian,brain}/status	each layer	all	1	yes

B HMAC reference implementation and test vector

B.1 Python

```
import hmac, hashlib, json

def seal(body: dict, key: bytes) -> str:
    text = json.dumps(body, separators=(",", ":"))
    tag = hmac.new(key, text.encode("utf-8"), hashlib.sha256).hexdigest()
    return json.dumps({"v": 1, "body": text, "hmac": tag})

def open_sealed(payload: str, key: bytes) -> dict:
    env = json.loads(payload)
    text = env["body"]
    expected = hmac.new(key, text.encode("utf-8"), hashlib.sha256).hexdigest()
    if not hmac.compare_digest(expected, env.get("hmac", "")):
        raise ValueError("E_AUTH")
    return json.loads(text) # then check timestamp, id, format, ids, range
```

B.2 Test vector

Every implementation must give exactly this result.

Key (UTF-8 text)	`test-secret-twin-brain`
Body text	see below
Expected HMAC	`c97ecfbc4f3f1c82140c3bd78d804b33 11af0bf051d5c98f025aef9f76b6771b`

```
{"correlation_id":"twin-000182","id":"twin-000182","sensor_id":"water-level-04","source
    ":"twin","timestamp":"2026-11-14T10:22:03.120Z","type":"water_level","unit":"m","
    value":2.4,"zone_id":"port"}
```

The key above is for testing only. Never use it in the real system.